G1 Report for TurtleBot Mission

Low Zhiyang	**A0299881A**
Jitisha Arora	**A0336857M**
Jane Shallum	**A0330815L**
Khaizuran Bin Khalid	**A0273597L**
Xu Huanghao	**A0306805H**

Introduction (Step 1: Problem Definition)

1.1 Mission Overview

This project requires the design of a TurtleBot-based autonomous mobile robot (AMR) that operates within a simulated warehouse maze to transport and deliver payloads represented by three ping pong balls. The robot must autonomously navigate an unknown environment, detect designated drop-off stations using chosen landmark detection methods (e.g., QR codes, RFID, or ArUco markers), and unload the payload into a receptacle (biscuit tin) at the correct location. The system should be designed for ease of operation while integrating navigation, perception, and delivery mechanisms into a cohesive autonomous solution.

1.2 Objectives

Our mission is to develop a TurtleBot3-based warehouse AMR that can autonomously navigate an unknown maze-like warehouse, detect landmarks, and deliver payloads which are 3 ping pong balls to designated stations.

- <u>Static delivery</u>
  Autonomously leave the start zone, explore/map the randomized maze, locate Station A using a visual marker, dock within tolerance, and unload the payload into the receptacle.
- <u>Dynamic delivery</u>
  Detect and track a moving delivery platform mounted on a linear rail, identify its landmark, time the docking manoeuvre, and successfully unload a payload batch into the moving receptacle.
- <u>Bonus Objective</u>
  Navigate to the lift lobby, initiate lift operation via API calls, enter safely, ascend to Level 2, exit, navigate to the final delivery point, and complete the final unloading sequence.

1.3 Constraints

- Time Constraint: The entire mission, including setup and execution, must be completed within the allocated 25-minute window.
- Autonomy Constraint: Once the mission begins, manual intervention or teleoperation is strictly prohibited.
- No Line Tracing: Line-following or line-tracing strategies are explicitly disallowed.
- Marker Deployment Constraint: Teams are allowed a maximum of six temporary markers, and all markers must be installed and removed within the mission time.

1.4 System Requirements (Step 1: Requirements Definition)

Functional Requirements:

- Autonomous maze navigation using LiDAR-based SLAM
- Landmark detection (QR / ArUco / RFID) at delivery stations
- Precise docking or positioning before delivery
- Controlled deployment of 3 ping pong balls into receptacle
- Obstacle avoidance during navigation
- Fully autonomous operation (no teleoperation)

Performance Requirements:

- No wall collisions during navigation
- Accurate marker detection and decoding
- Reliable distance measurement for launch positioning
- Mission completion within 25 minutes
- Stable operation without system crash or command delay

Literature Review (Step 2: Research)

2.1 Navigation (Software)

For autonomous navigation through the randomised maze and delivery zones, we would be utilising existing software packages within the Turtlebot framework such as Cartographer or Gmapping along with the existing odometry and LIDAR array available on the Turtlebot to perform Simultaneous Location and Mapping (SLAM). The occupancy grid generated from SLAM will allow the robot to localize itself within the maze and make navigation decisions based on both sensor data and mapped information.

We can implement custom navigation algorithms by building on the code provided to us in auto_nav, using key information such as LiDAR distance arrays, odometry feedback, and velocity control via /cmd_vel. Alternatively we can consider using nav2, one of the most popular navigation packages in ROS2. The following navigation approaches were evaluated:

1. Wall-following navigation
This is a rule-based reactive navigation method in which the robot maintains a fixed distance from a wall, typically on the left or right side, using LiDAR measurements. This can be combined with a PID controller to maintain a constant lateral distance from the wall. This method is computationally lightweight and is easy to implement. However, wall-following is not guaranteed to find the shortest path and can result in inefficient traverse in complex settings.

2. Frontier-based exploration
This utilises the occupancy grid generated from SLAM to identify boundaries between explored free space and unexplored regions, known as 'frontiers'. The robot selects frontier cells as temporary goals and navigates toward them, progressively expanding the known map. This method ensures a very systematic exploration and it also supports map closure, which is important in a randomised maze environment. As compared to purely reactive strategies, frontier exploration provides a rather structured coverage of unknown areas. However, this approach depends on very reliable localisation and mapping. Inaccuracies in the occupancy grid may lead to incorrect frontier detection.

3. A* Global path planning
A* is a graph-based search algorithm that computes the shortest path between a start and goal position on a discretised grid. Using the occupancy grid map, each free cell is treated as a node, and the algorithm evaluates paths based on accumulated cost and a heuristic estimate of remaining distance. A* guarantees optimality if the heuristic is admissible and is efficient in structured environments such as grid-based mazes. Once delivery station coordinates are known, A* can generate a reliable path to the target. The main limitation of A* is that it assumes a relatively static and known map. Hybrid approaches combining A* with local planners like DWA have been shown to overcome this limitation in dynamic environments.

4. Artificial Potential Fields (APF)
This models the robot as being influenced by 'virtual forces'. The goal exerts an attractive force pulling the robot toward it, while obstacles exert repulsive forces pushing the robot away. The resultant force vector determines the robot's steering direction and velocity. This method operates in continuous space and provides smooth, reactive

obstacle avoidance without requiring full path computation. However, APF is prone to local minima, where attractive and repulsive forces cancel out, potentially trapping the robot.

5. Dynamic WIndow Approach (DWA)
SWA is a local motion planning algorithm that computes velocity commands for robots in real time, considering the robot's kinematic constraints and obstacle proximity. At each time step, DWA samples a set of feasible linear and angular velocities within the robot's dynamic limits and evaluates them based on a cost function that typically combines proximity to the goal, distance from obstacles, and speed efficiency. The command with the lowest cost is selected for execution, allowing the robot to navigate safely while following a desired path. However, it requires computational resources to evaluate the velocity space continuously and is dependent on accurate obstacle detection. Recent improvements to DWA have addressed its limitations with dynamic obstacles by incorporating velocity change evaluation and safety distance coefficients.

2.2 Sensors and Camera (Electrical)

We consider using the TurtleBot3's built-in 360° Laser Distance Sensor (LDS-01) LiDAR for obstacle and target detection. When the LiDAR detects an object within a critical threshold (e.g., 10 cm) directly in front of the robot, the control logic triggers the ball-shooting mechanism. The LDS-01 has a sensing range of approximately 0.12 m to 3.5 m and provides continuous distance data suitable for both SLAM and proximity detection.

As an alternative, an ultrasonic sensor may be used to estimate distance before firing. Ultrasonic sensing is cost-effective and simple to integrate, but generally offers lower precision in cluttered environments compared to LiDAR's finer angular resolution.

For visual identification, a Raspberry Pi Camera Module is used to detect and decode visual markers placed within the maze. Video frames are processed in real time using OpenCV, while a decoding library such as ZBar identifies QR codes (or alternatively, ArUco markers). Once a marker is detected, its position relative to the center of the image frame is calculated to determine alignment before shooting. Bounding boxes are drawn to confirm detection and orientation. Sensor fusion between the camera, ultrasonic sensor, and LiDAR improves bin detection accuracy and overall launch precision.

2.3 Launch Mechanism (Mechanical)

There are various ways to implement a launch mechanism to fire projectiles into the bins. Two of the options in consideration are provided below:

Flywheel: The projectiles are loaded into a tube and are gravity-fed into the loading chamber. The motor of the flywheel is turned on, speeding up the flywheel. The motor for the feeding arm is turned on, which moves the feeding arm in translational, oscillating motion in the axis of the trajectory of fire. The friction between the flywheel and the projectile causes the kinetic energy in the flywheel to be transferred into the kinetic energy of the projectile, launching it.

Solenoid: A solenoid is mounted at the end of a barrel. It will constantly be activated to compress the spring in its mechanism. This spring will be used to transfer the force to the projectile. When the projectile is ready to be fired, the solenoid will be de-energised, releasing the spring to propel the projectile.

2.4 Reloading Mechanism (Mechanical)

There are not many reloading mechanisms within the context of firing projectiles along a horizontal trajectory.

Gravity fed: A tube spirals around the turtlebot above the launch mechanism. However, a gate mechanism is

needed to prevent double feeding of the projectiles and prevent jamming. When the gate opens, the next projectile rolls into the launch mechanism by gravity.

Individual launchers: There could be individual barrels with individual launch mechanisms. This does not rely on gravity but will need individual algorithms for each individual barrel.

Conceptual Design (Step 3: Concept Design)

Concept 1: A camera to scan AruCo codes, LIDAR for navigation and an ultrasonic sensor to accurately position the turtlebot before launching through a flywheel mechanism. The camera is used to scan the AruCo markers that mark where the bins are located for the projectiles. For navigation, the robot primarily uses wall-following navigation to traverse the maze incorporated with a PID controller. An ultrasonic sensor is used to measure the distance from the bin. When the distance measured is optimal, the launcher algorithm will input the speed for the flywheel mechanism to fire the projectile. A servo motor is used as the gate mechanism to release the projectile into the launcher mechanism. The robot will be using local APF to avoid obstacles.

Concept 2: A camera to scan QR codes, LIDAR for navigation and to accurately position the turtlebot before launching through a solenoid mechanism. The camera is used to scan the QR codes that mark the bins for the projectiles.The robot uses frontier-based exploration to expand the occupancy grid map until bins are found. It then uses A* global path planning to compute the shortest collision-free path towards the bin. The LIDAR sensor is used to measure the distance from the bin. Calculations and testing are done beforehand to measure the distance travelled by the projectile when fired by the solenoid. When the distance between the bin and turtlebot is optimal, the solenoid is actuated to fire the projectile out of the barrel. The robot will be using local DWA to avoid obstacles.

Concept Evaluation (Step 4: BOGAT/Brainstorming)

4.1 Trade-offs:

Based on the above concepts as summarised:

Subsystem	Concept 1	Concept 2
Software (Navigation)	Wall following navigation + PID + local APF	Frontier exploration + A* + local DWA
Electrical (Sensors)	LIDAR for navigation, ultrasound for positioning before launch, AruCo markers	LIDAR for navigation, LIDAR for positioning before launch, QR markers
Mechanical (Launching)	Flywheel	Solenoid

Here is our tradeoff considerations across each discipline after brainstorming:

Feature	**Advantage**	**Disadvantage**	**Evaluation method**

Flywheel launcher	Controlled firing distance	High power draw	Fire at different distances to evaluate firing algorithm
Solenoid Mechanism	Simple actuation, fast trigger response	performance affected by voltage drop	Test delivery success rate

Feature	Advantage	Disadvantage	Evaluation method
Ultrasonic sensor for positioning the turtlebot at launch position	Accurate distance measurement	Additional power draw	Compare with LIDAR distance values
LIDAR sensor for positioning the turtlebot at launch position	No extra hardware; stable distance readings from LDS-01	Minimum measurable distance	Compare LiDAR-reported distance to measured distance at launch position
AruCo markers	Controlled firing distance	High power draw	Accurately position the turtlebot to launch the balls
QR codes	Controlled firing distance	More difficult to detect as compared to AruCo	Accurately position the turtlebot to launch the balls

Feature	Advantage	Disadvantage	Evaluation method
Wall-following navigation	Low computational complexity	Inefficient in finding shortest path, lacks global awareness	Time taken to reach complete maze
Frontier-based exploration + A* global planning	Systematic exploration, faster finding of shortest path	Require more CPU, and accurate SLAM data	Time taken for projectile delivery and navigation
Artificial Potential Fields (APF)	Smooth obstacle avoidance	Limited global knowledge	Numbers of obstacles avoided
Dynamic Window Approach (DWA)	Reacts to newly discovered obstacles in real time	Dependent on accurate obstacle sensing	Numbers of obstacles avoided

4.2 Risks and Evaluations:

General System Risks:

a) System Crashes or Software Failures
   i) **Risk:** The ROS system may crash due to connection errors, CPU overloads and software bugs. Such failures could lead to unsafe behaviour during runs.
   ii) **Evaluation:** Implement hardware or software watchdogs that trigger E-Stop and restarts affected nodes if necessary.
b) Sensor or communication failure
   i) **Risk:** Sensors may disconnect or produce corrupted data.
   ii) **Evaluation:** Simulate sensor dropouts and packet loss and verify the fallback behaviour.

Navigation Risks:

a) Computational overload
   i) **Risk:** Running SLAM, global and local navigation algorithms simultaneously may strain RPi, causing delays in motion and control commands.
   ii) **Evaluation:** Monitor CPU and memory performance during navigation tasks, measure responsiveness of obstacle avoidance.
b) Localisation drifts and errors
   i) **Risk:** SLAM may produce incorrect maps due to sensor noise or odometry errors leading to incorrect path planning or missed targets.
   ii) **Evaluation**: Run multiple localisation trails on the same map and test loop-closure performance and map consistency .

Payload Deployment Risks:

a) Incorrect positioning due to sensor errors or calibration issues
   i) **Risk:** Inaccurate position and estimation from marker detection or ultrasonic sensors that cause inconsistent positioning.
   ii) **Evaluation:** Perform repeated positioning tests at different distances and angles, and track the optimal projectile delivery configuration and make adjustments accordingly.

4.3 Selected Concept:

The selected concept integrates the strongest elements from both conceptual designs.

Navigation System
The robot will implement a hierarchical navigation framework consisting of exploration, global path planning, and local obstacle avoidance layers. Initially, frontier-based exploration will be used to expand the occupancy grid map generated via SLAM. Once a bin location is detected through visual markers, A* global path planning will be applied to compute the shortest collision-free path from the robot's current pose to the target location on the occupancy grid. For local motion control and dynamic obstacle avoidance, the Dynamic Window Approach (DWA) will be employed. This hierarchical approach of A* for global planning and DWA for local avoidance have shown improvements in path efficiency and obstacle avoidance capability. LiDAR data will serve as the primary sensor for mapping, obstacle detection, and local planning.

Bin Detection and Sensor Fusion

Precise positioning before projectile deployment will be achieved through sensor fusion between the Raspberry Pi camera and an ultrasonic sensor. The camera will detect ArUco markers placed on the bins, providing relative pose

estimation for alignment and orientation correction. Once the robot is properly aligned, an ultrasonic sensor mounted at the front of the robot will measure the straight-line distance to the bin. This short-range measurement provides an additional layer of precision for determining the optimal firing distance. By combining vision-based pose estimation with ultrasonic distance validation, the system improves reliability while minimising computational overhead.

Launching Mechanism

The flywheel mechanism was selected for its precision and simplicity. With the speed control of a motor, the flywheel mechanism can be adjusted to different distances with an algorithm that uses the input from the distance detection system to calculate the distance and speed required. Since the flywheel can be switched off when not ready to fire, it can conserve power. This mechanism provides a robust solution for controlled firing of the projectile.

System Coordination via a Finite State Machine

To ensure structured and conflict-free operation, the overall system will be coordinated using a FSM. The FSM will manage high-level behavioral transitions between exploration, target detection, path planning, approach, alignment, firing, and recovery states. At any given time, only the active state will control motion commands, preventing interference between global planning, local obstacle avoidance, and launching routines. State transitions will be triggered by sensor inputs such as marker detection, distance thresholds, successful projectile deployment, or timeout conditions.

Preliminary Design (Step 5: Prototyping Plan)

Component Integration:

Turtlebot Base: Serves as the primary mobile platform, with onboard computing via Raspberry Pi, LiDAR sensing, and battery power supply. The TurtleBot handles SLAM, navigation, and high-level coordination through ROS2.
LiDAR system: Mounted centrally on the TurtleBot, providing 2D range data for SLAM and obstacle detection. Integrated with ROS2 Cartographer for real-time occupancy grid mapping. LiDAR data supports frontier-based exploration, A* global planning, and DWA local obstacle avoidance.
Vision System (ArUco Detection): A Raspberry Pi camera is mounted at the front of the TurtleBot to detect ArUco markers placed on bins. The camera provides relative pose estimation for target identification and alignment prior to projectile deployment.
Ultrasonic distance sensor: A forward-facing ultrasonic sensor is integrated to measure short-range distance from the bin during the final alignment phase. This sensor refines positioning accuracy before firing.
Flywheel Launch mechanism: A motor-driven flywheel launcher is mounted at the front of the TurtleBot. A servo-controlled gate mechanism regulates projectile release into the spinning flywheel. Motor speed is controlled via PWM to adjust projectile range based on measured distance.

Hardware Architecture:

Microcontroller Integration: The Raspberry Pi manages LiDAR, camera processing, and navigation logic. The ultrasonic sensor and servo motor are interfaced through GPIO.
Power Management: The TurtleBot battery powers the onboard computer and LiDAR. A regulated power distribution system supplies stable voltage to the flywheel motor and servo mechanism. Battery levels will be verified prior to each run.
Mechanical Design: Custom 3D-printed mounts are designed to securely position the camera, ultrasonic sensor, and flywheel launcher. The launcher assembly is aligned with the robot's forward axis to ensure consistent projectile trajectory.

Software Architecture:

ROS2 Framework: ROS2 manages communication between SLAM, exploration, planning, perception, and launching modules. Custom nodes will handle navigation control, marker detection, and actuator commands.
Exploration and Planning Module: Frontier-based exploration identifies unexplored regions within the occupancy grid. Once a bin marker is detected, A* computes the shortest collision-free path. DWA provides real-time local obstacle avoidance and velocity control.
Finite State Machine (FSM): A FSM coordinates high-level system behavior, managing transitions between exploration, path planning, alignment, firing, and recovery states. Only one state controls motion commands at a time to prevent algorithm conflicts.
Projectile Control Algorithm: A calibrated mapping function determines flywheel speed based on measured bin distance. When the optimal firing range is reached, the servo gate actuates to release the projectile into the flywheel mechanism.

Interfaces:

To ensure smooth subsystem integration, the following interfaces will be implemented:
ROS Nodes for Navigation and Perception: Custom ROS nodes subscribe to `/scan`, `/odom`, `/map`, and camera topics. Sensor data is processed and shared across modules via ROS topics, enabling coordination between exploration, planning, and firing systems.
Sensor Fusion Interface: The ArUco pose estimation provides alignment and coarse distance estimation, while ultrasonic readings refine short-range positioning. A validation check ensures stable distance measurements before projectile deployment.
Motor Control Interface: PWM control signals regulate flywheel speed, and GPIO commands actuate the servo gate. Feedback mechanisms monitor command execution timing to ensure consistent firing.

Verification Plan:

A comprehensive verification plan will be executed to ensure system performance:

Measures of Performance (MOPs):
- ArUco markers must be reliably detected within 1 meters
- Ultrasonic distance readings must maintain ±2 cm accuracy within the firing range
- Flywheel speed calibration must result in projectile landing within the bin target area during static testing

Technical Performance Measures (TPMs):
- Mission success is defined as successfully locating and firing projectiles into designated bins within the allocated mission time
- Navigation performance will be evaluated based on successful map closure and collision-free traversal of the maze.

Failure Modes:
Potential failure modes and mitigation strategies include:
- Localization Drift: If SLAM accuracy degrades, the system will reduce speed and rely on local obstacle avoidance while attempting map correction
- Marker Detection Failure: If ArUco detection fails, the robot will continue frontier exploration until the target is reacquired
- Ultrasonic Noise: Distance readings will be averaged across multiple samples before firing to reduce noise-induced errors
- Launcher Malfunction: If projectile deployment fails, the FSM transitions to a recovery state to retry alignment and firing

Conclusion & Next Steps

The proposed TurtleBot system integrates structured exploration, optimal path planning, precise alignment, and

controlled projectile deployment within a layered navigation framework. Frontier-based exploration ensures systematic maze coverage, while A* and DWA provides efficient and safe navigation toward detected bins. The flywheel launching mechanism offers adjustable and repeatable projectile control, supported by vision-based alignment and ultrasonic distance refinement.

Moving forward, we will finalize mechanical mounting designs, complete flywheel speed calibration experiments, and conduct integrated system testing under simulated maze conditions. The next steps include refining the FSM logic, optimizing sensor thresholds, and preparing for Preliminary Design Review (PDR).

Appendix

References

[1] A. Stentz, “Optimal and efficient path planning for partially-known environments,” *Proc. IEEE Int. Conf. Robotics and Automation*, 1994. [Online]. Available: https://dl.acm.org/doi/10.1145/280765.280773. [Accessed: 19-Feb-2026].

[2] B. Yamauchi, “A frontier-based approach for autonomous exploration,” 1997. [Online]. Available: https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/integrated1/yamauchi_frontiers.pdf. [Accessed: 19-Feb-2026].

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. [Online]. Available: https://people.stfx.ca/jdelamer/courses/csci-564/_downloads/b2220c66675ddde471ca1795147b8e86/A_Formal_Basis_for_the_Heuristic_Determination_of_Minimum_Cost_Paths.pdf. [Accessed: 19-Feb-2026].

[4] Cytron Technologies, “Reading QR code using Raspberry Pi camera,” Cytron.io, 2023. [Online]. Available: https://www.cytron.io/tutorial/reading-qr-code-using-raspberry-pi-camera. [Accessed: 19-Feb-2026].

[5] Circuit Digest, “QR code scanner using Raspberry Pi and OpenCV,” CircuitDigest.com, 2022. [Online]. Available: https://circuitdigest.com/microcontroller-projects/qr-code-scanner-using-raspberry-pi-and-opencv. [Accessed: 19-Feb-2026].

[6] ROBOTIS, “LDS-01 LiDAR sensor (TurtleBot3),” ROBOTIS e-Manual, 2023. [Online]. Available: https://emanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_01/. [Accessed: 19-Feb-2026].

[7] “EV3 Projectile Launcher Informational Video,” *YouTube*, 2020. [Online]. Available: https://www.youtube.com/watch?v=4risclUjeXg. [Accessed: 19-Feb-2026].

[8] “Table Tennis Robot Mechatronics,” *YouTube Shorts*, 2023. [Online]. Available: https://www.youtube.com/shorts/AhTZ4MJgbOc. [Accessed: 19-Feb-2026].

[9] “Dog Ball Launcher,” *YouTube*, 2021. [Online]. Available: https://www.youtube.com/watch?v=s4MhGgzHkAo. [Accessed: 19-Feb-2026].

[10] Y. Cao and N. M. Nor, “An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation,” Decision Analytics Journal, vol. 11, Jun. 2024, Art. no. 100471. doi: 10.1016/j.dajour.2024.100471.

[11] Y. Zhao, P. Li, and Y. Liu, "Robot obstacle avoidance path planning by integrating improved A* algorithm and improved dynamic window approach," PLoS ONE, vol. 19, no. 5, May 2024, Art. no. e0302026. doi: 10.1371/journal.pone.0302026.